Aim

To design and implement a **Question Answering (QA) system** using a fine-tuned BERT model that extracts relevant answers from a given context based on user queries.

Theory

Question Answering (QA) is a Natural Language Processing (NLP) task where a system automatically responds to user queries in natural language. One powerful approach is using **BERT (Bidirectional Encoder Representations from Transformers)**, developed by Google.

BERT is based on the **Transformer architecture**, which uses self-attention mechanisms to understand the contextual relationships between words in a sentence. Unlike traditional models, BERT reads text **bidirectionally**, meaning it considers both left and right context simultaneously.

In QA tasks (like the SQuAD dataset), BERT is fine-tuned to predict:

* **Start index** of the answer in the context
* **End index** of the answer in the context

The model outputs probability distributions over tokens, and the most probable span is selected as the answer.

There are two types of QA:

* **Extractive QA** (used here): Answer is extracted from context

* **Generative QA**: Answer is generated (like GPT models)

Procedure

Step 1: Install Required Libraries

```bash
pip install transformers torch
```

Step 2: Import Libraries

```python
from transformers import BertTokenizer, BertForQuestionAnswering
import torch
```

Step 3: Load Pre-trained Fine-tuned Model

```python
model_name = "bert-large-uncased-whole-word-masking-finetuned-squad"

tokenizer = BertTokenizer.from_pretrained(model_name)
model = BertForQuestionAnswering.from_pretrained(model_name)
```

Step 4: Define QA Function

```python
def get_answer(question, context):
    inputs = tokenizer.encode_plus(question, context, return_tensors="pt")

    outputs = model(**inputs)
    start_scores = outputs.start_logits
    end_scores = outputs.end_logits

    start_index = torch.argmax(start_scores)
    end_index = torch.argmax(end_scores) + 1

    answer = tokenizer.decode(inputs["input_ids"][0][start_index:end_index])
    return answer
```

Step 5: Provide Input

```python
context = "BERT is a transformer-based model used for NLP tasks."
question = "What is BERT used for?"
```

Step 6: Get Output

```python
answer = get_answer(question, context)
```

```
print("Answer:", answer)
```

Conclusion

A Question Answering system using a fine-tuned BERT model was successfully implemented. The system efficiently extracts accurate answers from a given passage by predicting the start and end positions of the answer span. This demonstrates the effectiveness of transformer-based models in solving real-world NLP tasks such as information retrieval and conversational AI.